

Week 10 - Image Classification on MNIST - From Handcrafted Features to Deep Learning

[15/Apr/2026 Dr Min Wang]

Overview

Image classification is a fundamental task in computer vision, where the goal is to assign a label to an input image. Traditionally, this problem relied on handcrafted feature extraction combined with classical machine learning models. More recently, deep learning approaches have enabled automatic feature learning directly from raw data.

In this lab, you will explore both paradigms using the MNIST handwritten digit dataset. You will implement three different pipelines and compare their performance:

- HOG + SVM: Edge-based handcrafted features
- SIFT + SVM: Local invariant features with Bag-of-Visual-Words
- CNN: End-to-end deep learning model

This lab is designed to help you understand how feature representation impacts classification performance and model design.

Learning Objectives

By the end of this lab, you should be able to:

- Understand the difference between handcrafted features and learned features
- Extract HOG and SIFT features from images
- Apply Support Vector Machines (SVMs) for classification
- Design and train a Convolutional Neural Network (CNN)
- Evaluate models using:
 - Accuracy
 - Confusion Matrix
- Critically compare traditional and deep learning approaches

Required Packages

We need the following libraries, you should have been familiar with them by now.

- numpy: array processing for numbers, strings, records and objects
- opencv (cv2): computer vision and machine learning library
- matplotlib: figures in python

Install them if you haven't done so: `pip install numpy opencv-python matplotlib scikit-learn`

In addition, we will need the following two today:

- tensorflow: an end-to-end open source platform for machine learning
- scikit-learn (sklearn): a set of python modules for machine learning and data mining
- scikit-image (skimage): image processing in python

Install them: `pip install tensorflow scikit-learn scikit-image`

Be careful about the version compliance on your machine.

```
In [66]: # OK, now let's verify Installation
# Run these commands to ensure both are installed correctly.
import tensorflow as tf
print('tensorflow version: ', tf.__version__)

import sklearn
print('scikit-learn version: ', sklearn.__version__)
```

```
tensorflow version: 2.16.2
scikit-learn version: 1.7.2
```

Dataset Description

We will use the MNIST dataset, which consists of:

- 70,000 grayscale images of handwritten digits (0–9)
- Image size: 28 × 28 pixels
- Training set: 60,000 images
- Test set: 10,000 images

1. Load Dataset and Exploration

```
In [67]: # =====  
# Load MNIST dataset  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
from tensorflow.keras import datasets  
  
# Load dataset  
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()  
# Normalise pixel values to [0,1]  
x_train = x_train / 255.0  
x_test = x_test / 255.0
```

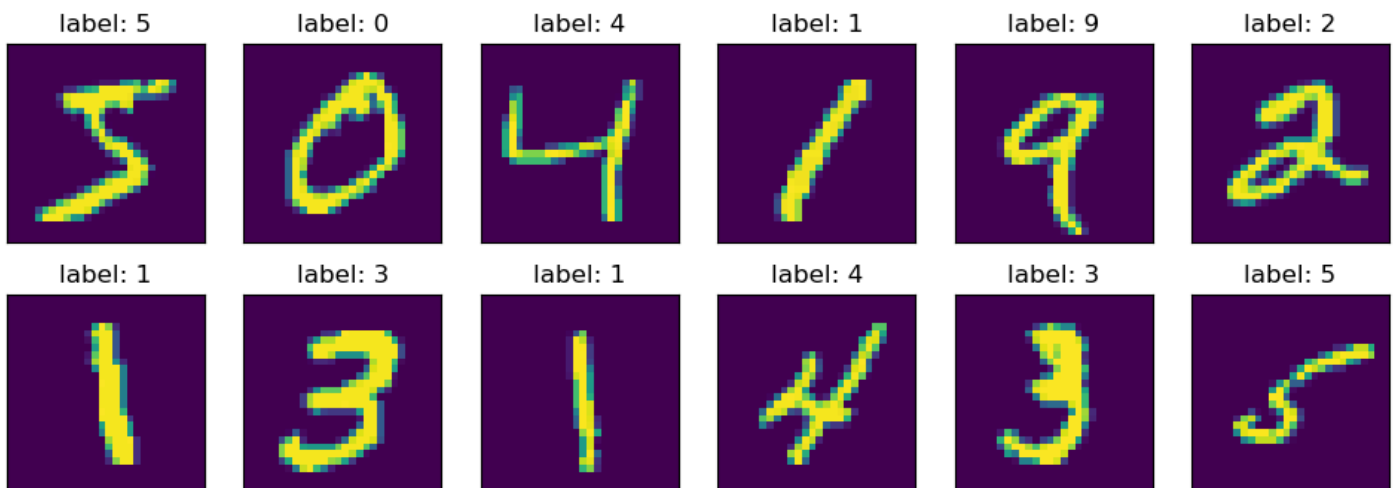
```
In [68]: # Now before moving on, let's explore the dataset and visualise the data  
  
# Find number of classes (unique numbers of the train labels)  
classes = np.unique(y_train)  
print('Number of classes: ', len(classes))  
print('Output classes: ', classes)  
  
# Find number of training data and data shape  
x_train.shape
```

Number of classes: 10

Output classes: [0 1 2 3 4 5 6 7 8 9]

Out[68]: (60000, 28, 28)

```
In [69]: # Visualise some images  
plt.figure(figsize=(12, 4))  
for i in range(12): # show 12 images  
    plt.subplot(2, 6, i + 1)  
    plt.xticks([])  
    plt.yticks([])  
    plt.grid(False)  
    plt.imshow(x_train[i])  
    plt.title('label: '+str(y_train[i])) # label  
plt.show()
```



2. HOG Features + SVM

Histogram of Oriented Gradients (HOG) captures the shape and structure of objects by computing gradient orientations in localised regions. It is particularly effective for capturing edge information.

In this task, we will:

- Extract HOG features from images
- Train an SVM classifier
- Evaluate classification performance

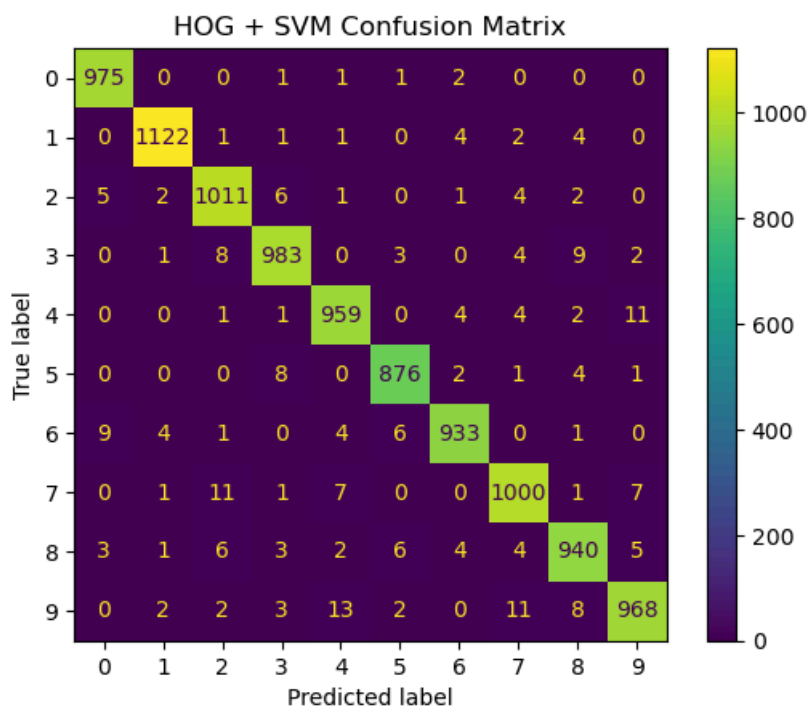
```
In [84]: # =====  
# Extract HOG Features  
# =====  
  
from skimage.feature import hog  
  
# Define our HOG feature extraction function  
def extract_hog(images):  
    features = []  
    for img in images:  
        hog_feat = hog(  
            img,  
            orientations=9,  
            pixels_per_cell=(7, 7),  
            cells_per_block=(2, 2),  
            block_norm='L2-Hys'  
        )  
        features.append(hog_feat)  
    return np.array(features)  
  
# Extract HOG features for the train and test sets  
X_train_hog = extract_hog(x_train[:])  
X_test_hog = extract_hog(x_test[:])
```

```
In [85]: # =====  
# Train SVM Classifier  
# =====  
  
# We'll use a linear kernel for efficiency, but feel free to use non-linear kernels in your own code  
  
from sklearn import svm  
  
clf_hog = svm.SVC(kernel='linear')  
clf_hog.fit(X_train_hog, y_train)
```

```
Out[85]: SVC ⓘ ?  
► Parameters
```

```
In [86]: # =====  
# Evaluate Performance  
# =====  
  
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay  
import matplotlib.pyplot as plt  
  
# Use the trained SVM classifier for prediction on test set  
y_pred_hog = clf_hog.predict(X_test_hog)  
  
# Compute and print accuracy  
print("HOG + SVM Accuracy:", accuracy_score(y_test, y_pred_hog))  
  
# Compute confusion matrix  
cm = confusion_matrix(y_test, y_pred_hog)  
ConfusionMatrixDisplay(cm).plot()  
plt.title("HOG + SVM Confusion Matrix")  
plt.show()
```

HOG + SVM Accuracy: 0.9767



3. SIFT + SVM (Bag of Visual Words)

SIFT detects keypoints and local descriptors that are invariant to scale and rotation. Since each image produces a variable number of descriptors, we use Bag of Visual Words (BoVW) to convert them into fixed-length feature vectors.

Pipeline:

- Extract SIFT descriptors
- Cluster descriptors (K-means) → visual vocabulary
- Represent each image as a histogram
- Train SVM

```
In [94]: # =====
# Extract SIFT Descriptors
# =====

import cv2

sift = cv2.SIFT_create()

# Define SIFT feature extraction function
def extract_sift(images):
    descriptors_list = []
    for img in images:
        img_uint8 = (img * 255).astype(np.uint8)
        _, descriptors = sift.detectAndCompute(img_uint8, None)
        if descriptors is not None:
            descriptors_list.extend(descriptors)
    return np.array(descriptors_list)

sift_descriptors = extract_sift(x_train[:10000])
```

```
In [95]: # =====
# Build Visual Vocabulary (K-Means)
# =====

from sklearn.cluster import MiniBatchKMeans

k = 50
kmeans = MiniBatchKMeans(n_clusters=k, random_state=0)
kmeans.fit(sift_descriptors)
```

```
Out[95]: ▼ MiniBatchKMeans ⓘ ⓘ
          ► Parameters
```

```
In [97]: # =====
# Convert Images to Histograms
# =====
```

```
# Define the function for BoVW representations
def build_histogram(images, kmeans):
    histograms = []
    for img in images:
        img_uint8 = (img * 255).astype(np.uint8)
        _, descriptors = sift.detectAndCompute(img_uint8, None)

        hist = np.zeros(k)
        if descriptors is not None:
            clusters = kmeans.predict(descriptors)
            for c in clusters:
                hist[c] += 1

        histograms.append(hist)
    return np.array(histograms)

X_train_sift = build_histogram(x_train[:10000], kmeans)
X_test_sift = build_histogram(x_test[:2000], kmeans)

y_train_sift = y_train[:10000]
y_test_sift = y_test[:2000]

# Note: here, again, we just use the first 10000 in training set and first 2000 in test set to speed up the con
# But you should perform the experiment using the whole dataset after the class by your own.
```

```
In [98]: # =====
# Train SVM and Evaluate
# =====

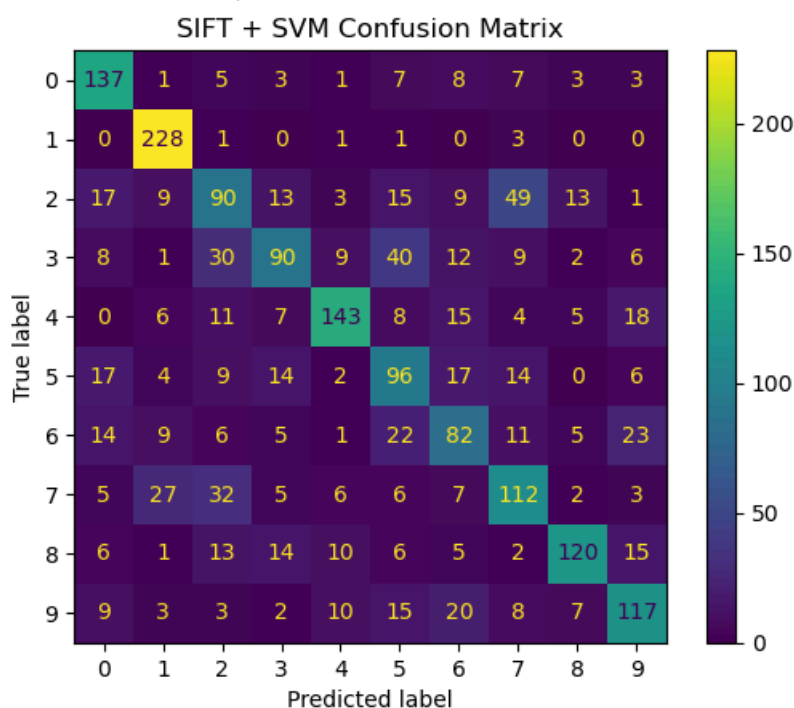
# Train an SVM (linear kernel)
clf_sift = svm.SVC(kernel='linear')
clf_sift.fit(X_train_sift, y_train_sift)

# Prediction
y_pred_sift = clf_sift.predict(X_test_sift)

# Accuracy
print("SIFT + SVM Accuracy:", accuracy_score(y_test_sift, y_pred_sift))

# Confusion matrix
cm = confusion_matrix(y_test_sift, y_pred_sift)
ConfusionMatrixDisplay(cm).plot()
plt.title("SIFT + SVM Confusion Matrix")
plt.show()
```

SIFT + SVM Accuracy: 0.6075



4. Deep Learning Approach (CNN)

Convolutional Neural Networks (CNNs) automatically learn hierarchical features. Unlike handcrafted features, CNNs learn directly from raw pixels in an end-to-end manner. A CNN typically consists of the following components:

- Convolutional Layers: These apply learnable filters (kernels) to the image to detect patterns such as edges, textures, and shapes.
 - Early layers learn simple features (edges)
 - Deeper layers learn complex patterns (digit shapes)
- Activation Functions (ReLU): Introduce non-linearity, allowing the network to learn complex relationships.
- Pooling Layers (Max Pooling): Reduce spatial dimensions (downsampling), making the model more efficient and robust to small variations.
- Fully Connected Layers: Combine extracted features to perform classification.
- Softmax Output Layer: Produces probabilities for each class (digits 0–9).

CNNs Work Well for Images, they:

- Automatically learn hierarchical representations
- Preserve spatial structure of images
- Require minimal manual feature engineering
- Achieve state-of-the-art performance in vision tasks

```
In [99]: # =====
# Prepare the Data
# =====

# First, let's format the data correctly for CNN input. CNNs expect images with a channel dimension.
# MNIST images are grayscale, so we add the channel dimension

import numpy as np

# Add channel dimension: (28,28) → (28,28,1)
x_train_cnn = x_train[..., np.newaxis]
x_test_cnn = x_test[..., np.newaxis]

print("Training shape:", x_train_cnn.shape)
```

Training shape: (60000, 28, 28, 1)

Define the CNN Architecture.

In this example, we will build a model that can extract features and classify digits.

- Use 2 convolutional layers
- Add pooling after each convolution
- Use a fully connected layer before output

```
In [100]: # =====
# Define the CNN Architecture
# =====

from tensorflow.keras import layers, models

model = models.Sequential([
    # First convolution block
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),

    # Second convolution block
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    # Classification head
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
/opt/anaconda3/envs/tf/lib/python3.10/site-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Compile the Model

We need to configure how the model learns. Here, we need to specify:

- Optimizer: Adam
- Loss: sparse_categorical_crossentropy
- Metric: accuracy

```
In [101... model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 128)	204,928
dense_3 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

Train the model

This is to learn feature representations and classification. We will train the model for 5–10 epochs, and use validation split to monitor overfitting.

```
In [102... # =====
# Train the Model
# =====

history = model.fit(
    x_train_cnn,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1 # use 0.1 of train set as validation set to monitor overfitting
)
```

```
Epoch 1/5
844/844 ————— 8s 9ms/step - accuracy: 0.9477 - loss: 0.1759 - val_accuracy: 0.9857 - val_loss: 0.0559
Epoch 2/5
844/844 ————— 8s 9ms/step - accuracy: 0.9821 - loss: 0.0603 - val_accuracy: 0.9848 - val_loss: 0.0572
Epoch 3/5
844/844 ————— 8s 9ms/step - accuracy: 0.9852 - loss: 0.0496 - val_accuracy: 0.9883 - val_loss: 0.0476
Epoch 4/5
844/844 ————— 8s 9ms/step - accuracy: 0.9867 - loss: 0.0504 - val_accuracy: 0.9877 - val_loss: 0.0610
Epoch 5/5
844/844 ————— 8s 9ms/step - accuracy: 0.9861 - loss: 0.0581 - val_accuracy: 0.9887 - val_loss: 0.0737
```

Evaluate the Model

Here we measure performance on unseen test data

```
In [103... # =====
# Evaluate the Model
# =====

# Accuracy
test_loss, test_acc = model.evaluate(x_test_cnn, y_test)
print("CNN Test Accuracy:", test_acc)

# Predictions on test set
y_pred_cnn = np.argmax(model.predict(x_test_cnn), axis=1)
```

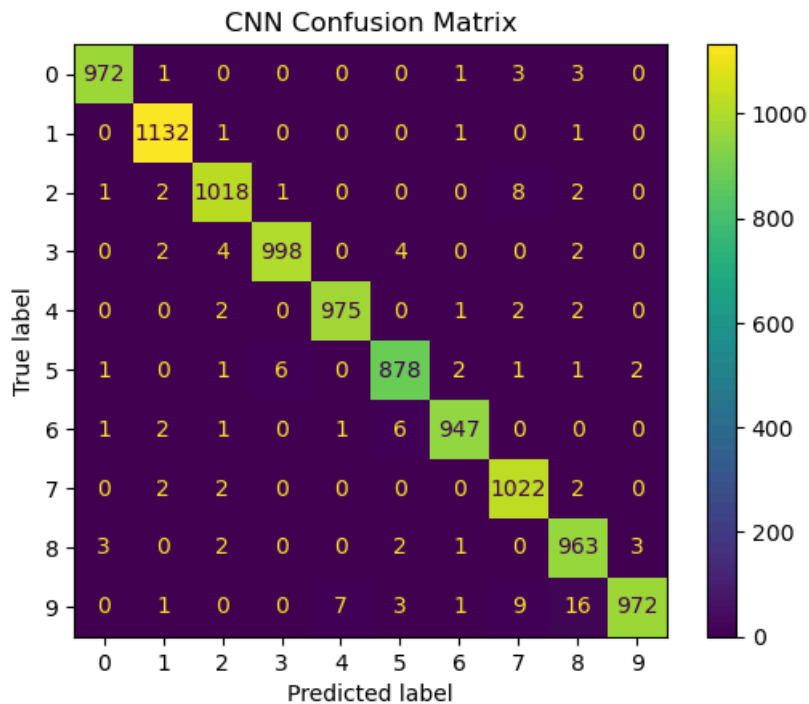
```
# Confusion matrix
cm = confusion_matrix(y_test, y_pred_cnn)

ConfusionMatrixDisplay(cm).plot()
plt.title("CNN Confusion Matrix")
plt.show()
```

313/313 ————— 1s 3ms/step – accuracy: 0.9877 – loss: 0.0584

CNN Test Accuracy: 0.9876999855041504

313/313 ————— 0s 1ms/step



Visualising misclassified images (incorrect predictions)

```
In [104... # Find indices where prediction is incorrect
incorrect_indices = np.where(y_pred_cnn != y_test)[0]

print("Number of incorrect predictions:", len(incorrect_indices))
```

Number of incorrect predictions: 123

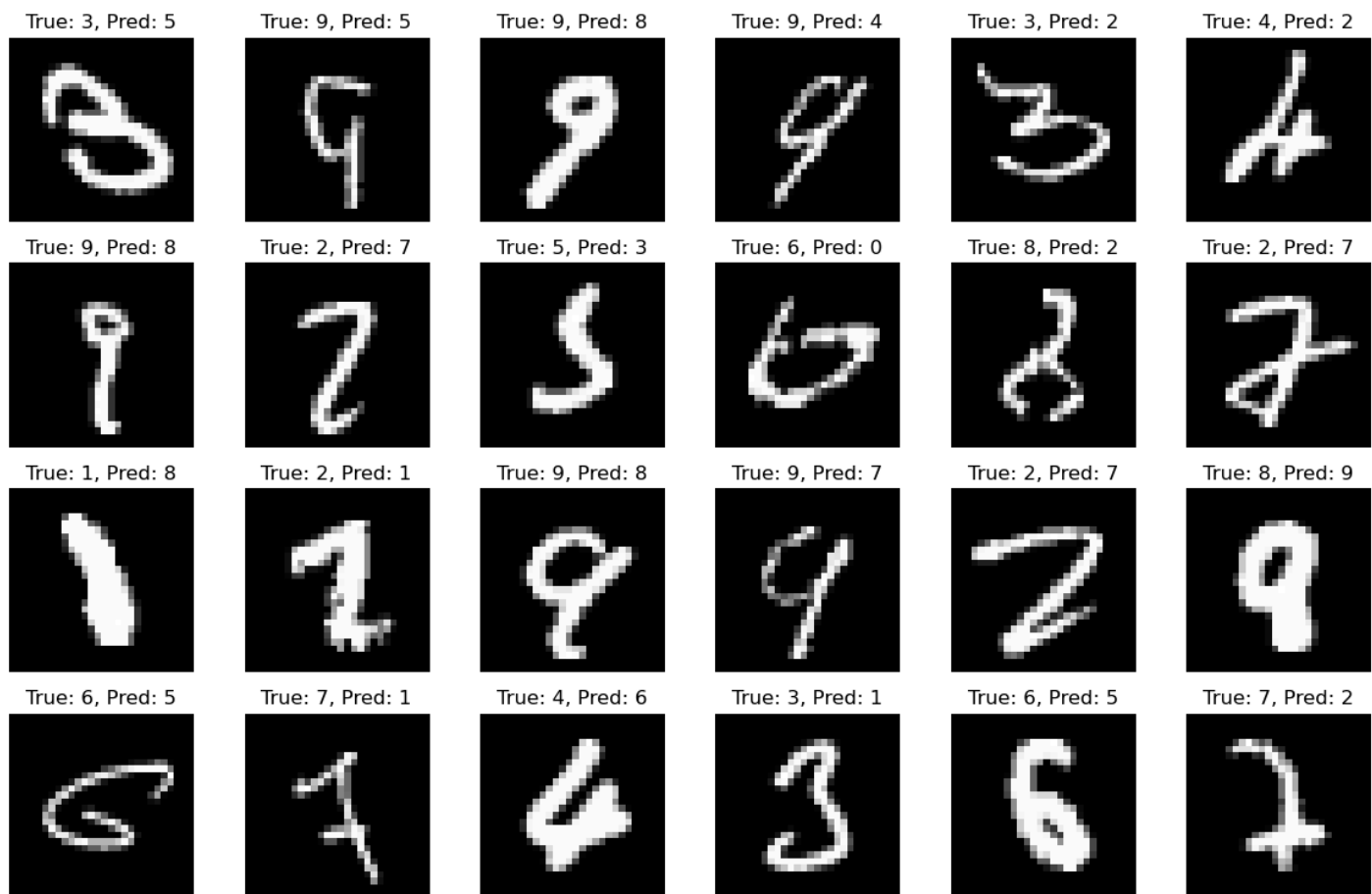
```
In [106... # Plot misclassified examples
plt.figure(figsize=(12, 8))

for i in range(24):
    idx = incorrect_indices[i]

    plt.subplot(4, 6, i+1)
    plt.imshow(x_test[idx], cmap='gray')
    plt.title(f"True: {y_test[idx]}, Pred: {y_pred_cnn[idx]}")
    plt.axis('off')

plt.suptitle("Incorrect Predictions (CNN)")
plt.tight_layout()
plt.show()
```


Incorrect Predictions (CNN)



Discussion

1. What type of information does HOG capture that makes it effective for digit recognition?
2. Why might SIFT perform poorly on MNIST images?
3. How does the Bag-of-Visual-Words approach help with SIFT features?
4. Why does the CNN outperform both HOG and SIFT approaches?
5. What are the advantages and disadvantages of:
 - Handcrafted features vs learned features?
 - SVM vs CNN?
6. From the confusion matrices:
 - Which digits are most frequently confused?
 - Why do these errors occur?
7. When might handcrafted features still be preferable over CNNs?
8. How would performance change if 1) The dataset were smaller? and 2) The images were higher resolution?

In []: